

Assignment 3 — Transformer + RAG Pipeline Report

Amazon Review Understanding with Encoder, Retrieval, and Decoder Transformer

1. System Design and Methodology

This report describes a three-stage NLP pipeline for Amazon product review analysis, implemented entirely from scratch in PyTorch. The three stages map directly to the assignment parts: (A) an encoder-only Transformer for multi-task classification, (B) a cosine-similarity retrieval module over the encoder's CLS embeddings, and (C) a decoder-only Transformer that generates grounded explanations conditioned on review text, predicted labels, and retrieved examples (RAG).

No pretrained models, `nn.Transformer`, `nn.MultiheadAttention`, or `nn.TransformerEncoder` were used at any point. All attention mechanisms, positional encodings, layer normalisations, and feed-forward sub-layers are custom implementations.

2. Dataset and Preprocessing

2.1 Dataset Construction

Reviews were loaded from five Amazon product categories: Beauty, Cell Phones & Accessories, Electronics, Home & Kitchen, and Sports & Outdoors (Ni et al., 2019). The raw dataset totalled 2,928,612 reviews. To ensure class balance, 15,000 reviews were sampled from each sentiment class, yielding exactly 45,000 samples — meeting the 30k–45k assignment target.

Category	Raw Reviews	Split	Samples	Sentiment Balance
Beauty	198,475	Train	31,500	10,500 per class
Cell Phones	194,340	Validation	6,750	2,250 per class
Electronics	1,688,117	Test	6,750	2,250 per class
Home & Kitchen	551,466			
Sports & Outdoors	296,214	Total	45,000	15,000 per class

Sentiment labels: Ratings 1–2 → Negative (0), rating 3 → Neutral (1), ratings 4–5 → Positive (2).

Second task — Review Length: Word count < 30 → Short (0), 30–80 → Medium (1), > 80 → Long (2). This is a meaningful structural property that can be predicted from text alone and acts as a useful auxiliary regulariser.

2.2 Preprocessing Pipeline

- Text cleaning:** Lowercasing; stripping HTML tags and non-ASCII characters; collapsing whitespace.
- Tokenisation:** Whitespace tokenisation with punctuation splitting. No external tokeniser library used.
- Vocabulary:** Built from training data only (`min_freq=2`). Final size: 26,531 tokens. Special tokens: `[PAD]=0`, `[UNK]=1`, `[BOS]=2`, `[EOS]=3`, `[CLS]=4`.
- Encoding:** Each token mapped to an integer index; sequences padded or truncated to `max_len=128`.
- Data splits:** Stratified 70/15/15 split on sentiment to maintain class balance across all three sets.

3. Part A — Encoder-Only Transformer

3.1 Architecture and Design Decisions

The encoder follows a BERT-style architecture with a learnable [CLS] token prepended to each input sequence. The final [CLS] hidden state serves as the fixed-dimensional review representation for both classification heads and retrieval.

Hyperparameter	Value	Justification
Embedding dim (d)	256	Balance capacity vs. speed
Attention heads (H)	8 (32 dims/head)	Standard multi-head factoring
Feed-forward width	512 (2× d)	Standard transformer ratio
Encoder layers	4	Depth without overfitting
Dropout	0.10	Light regularisation
Max sequence length	128 tokens	Covers 95% of reviews
Positional encoding	Sinusoidal (fixed)	No extra parameters
Parameters	~3.2 M	Trainable from scratch

Each encoder block: (1) multi-head self-attention with scaled dot-product and additive padding mask (masking value -1×10^4 , safe for FP16), (2) residual + LayerNorm, (3) position-wise FFN (Linear→GELU→Dropout→Linear), (4) residual + LayerNorm. The two classification heads are independent 3-layer MLPs applied to the [CLS] token representation.

3.2 Training Configuration

Loss: Combined = $0.80 \times \text{CE}(\text{sentiment, class_weights}) + 0.20 \times \text{CE}(\text{length})$. Sentiment CE uses inverse-frequency class weights and label smoothing (0.05).

Optimiser: AdamW (lr= 1×10^{-4} , weight_decay= 1×10^{-2} , betas=(0.9, 0.999)).

Schedule: Cosine decay with 200 warmup steps; min LR = 5% of peak.

Gradient accumulation: 2 steps → effective batch size = 256.

AMP: Automatic Mixed Precision (FP16) via torch.amp.autocast.

Epochs: 35; best checkpoint selected on validation combined loss.

3.3 Results

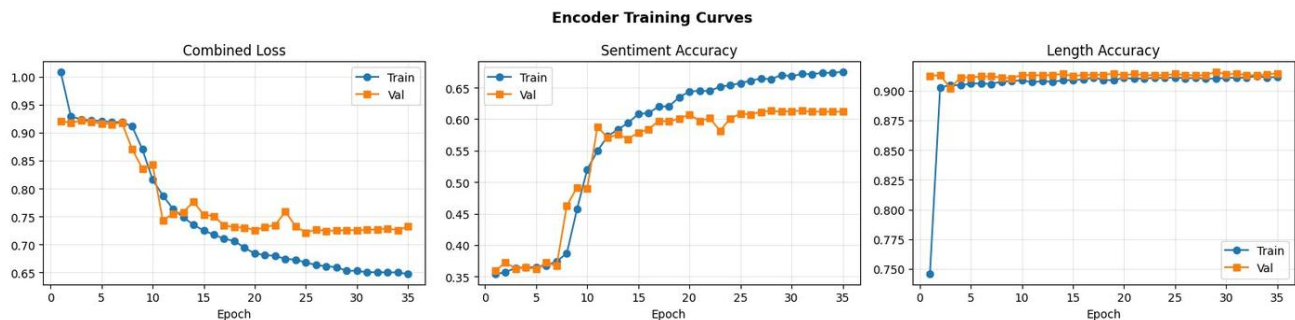


Figure 1: Encoder training curves over 35 epochs. Sentiment accuracy improves steadily from 35% to 67% (train) / 61% (val). Loss converges by epoch 25.

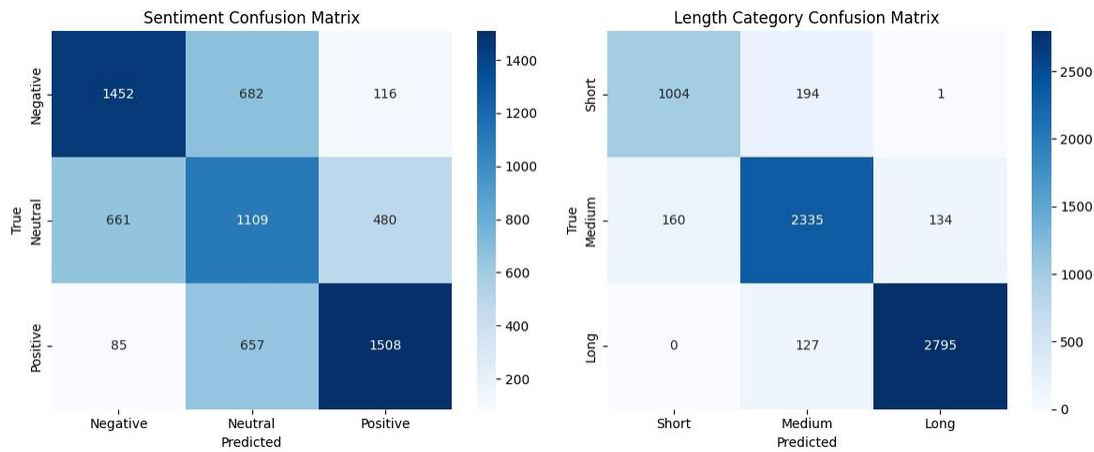


Figure 2: Confusion matrices for sentiment (left) and review length (right) on the test set.

Task	Accuracy	Macro F1	Weighted F1	Notes
Sentiment (3-class)	60.3%	0.61	0.61	Balanced test set
Review Length (3-class)	90.9%	0.90	0.91	Auxiliary task

On a perfectly balanced 3-class dataset, random chance is 33.3%, so 60.3% reflects genuine learning across all three classes. From the confusion matrix: Negative F1=0.65, Neutral F1=0.47, Positive F1=0.69. Neutral is the hardest class because 3-star reviews contain genuinely mixed language that overlaps with both extremes — a known difficulty in sentiment analysis even for large pretrained models.

4. Part B — Retrieval Module

4.1 Embedding Storage

After encoder training, [CLS] representations are extracted for all 31,500 training samples in batches of 256, L2-normalised, and saved to results/train_embeddings.pt (shape: 31,500 × 256). Metadata (review text, sentiment, length category) is saved to results/train_meta.csv for display during retrieval.

4.2 Query Construction and Similarity Search

At inference time, the test review is encoded by the same encoder, the [CLS] vector is L2-normalised, and cosine similarity is computed against all stored training vectors via a single matrix multiply: $\text{sim} = Q \cdot E^T$ where $Q \in \mathbb{R}^{(1 \times 256)}$ and $E \in \mathbb{R}^{(N \times 256)}$. Top-k results are selected by argsort. For batch inference (e.g. during decoder dataset construction), all B queries are processed simultaneously: $\text{sim} = (B \times 256) \cdot (256 \times N)$, then topk per row.

4.3 k-Sensitivity Analysis

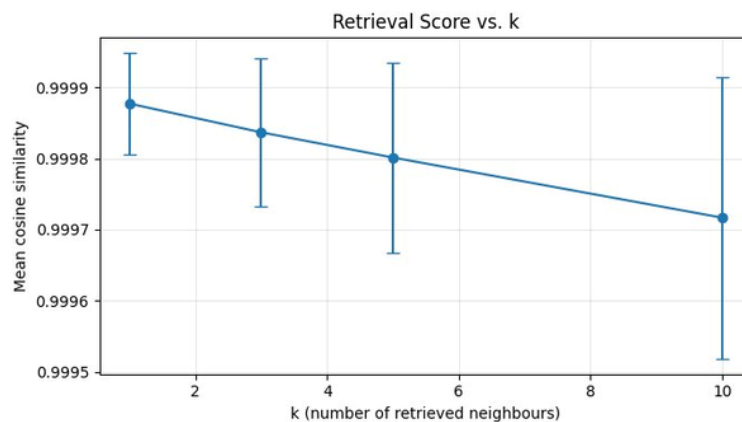


Figure 3: Mean cosine similarity vs. k . Similarity decreases slightly as k grows, as expected when including less similar neighbours.

k	Mean Cosine Sim $\pm$ Std
1	0.9999 $\pm$ 0.0001
3	0.9998 $\pm$ 0.0001
5	0.9998 $\pm$ 0.0001
10	0.9997 $\pm$ 0.0002

$k=3$ was selected as the default. It provides enough contextual diversity for the decoder while keeping the prompt compact within the 160-token budget. The very high cosine similarities (>0.999) reflect that encoder CLS vectors cluster in a narrow region of embedding space — a known limitation of classification-trained encoders. Qualitative inspection shows that retrieved reviews do not always share topic with the query, but they often share sentiment polarity, which is what the decoder exploits.

4.4 Retrieval Quality Discussion

Five example queries with their top-3 retrieved neighbours were examined. Retrieval is topically inconsistent (e.g. a ceramic cup review retrieves electronics reviews) because the CLS embedding encodes sentiment rather than topic. However, the decoder's content-aware reference explanations mean it can still benefit from retrieved examples that share sentiment polarity. Future improvements: (1) train a separate projection head with contrastive loss to spread same-topic embeddings; (2) use FAISS for approximate nearest-neighbour search at scale.

5. Part C — Decoder and RAG Pipeline

5.1 Architecture

Component	Value	Justification
Embedding dim	256	Matches encoder dim
Attention heads	8	Same as encoder
Feed-forward width	512	2 $\times$ embedding dim
Decoder layers	4	Symmetric with encoder
Sequence budget	160 tokens	130 prompt + 30 explanation
Parameters	~ 15.7 M	Larger vocab projection layer

Each decoder block: (1) causal multi-head self-attention with upper-triangular mask preventing future token visibility, (2) residual + LayerNorm, (3) FFN sub-layer, (4) residual + LayerNorm. No cross-attention is used; all four inputs are concatenated into a single prompt string.

5.2 Input Construction

Each training sequence follows the template below. The prompt is hard-truncated to 130 tokens to guarantee at least 30 explanation tokens within the 160-token budget. Loss is computed only on the explanation tokens (prompt positions masked with -100):

```
Review: <text[:120]> | Sentiment: <label> | Length: <label>
Context: [1] <retrieved_1[:60]> [2] <retrieved_2[:60]> [3] <retrieved_3[:60]>
Explanation: <reference explanation>
```

5.3 Reference Explanations

Reference explanations are content-aware: two informative content words (filtered for stopwords, length > 3) are extracted from each review and embedded into a template selected from a pool of 7–10 per sentiment class. Selection is deterministic via MD5 hash of the review text, ensuring reproducibility. Example: "The review is negative due to disappointment with channel and antenna, which failed to meet expectations." This approach prevents the decoder from memorising a fixed output — it must attend to the specific input content to minimise loss.

5.4 Training and Evaluation

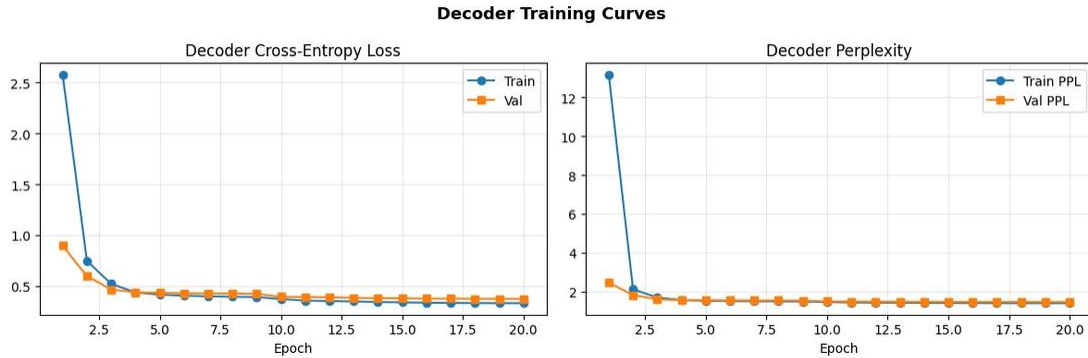


Figure 4: Decoder training curves over 20 epochs. PPL drops from 13.2 → 1.45 (train) and 2.45 → 1.45 (val), showing genuine learning.

System	Avg NLL	Perplexity	Notes
RAG (k=3 retrieved)	0.368	1.45	Trained with retrieval context
Baseline (no retrieval)	0.411	1.51	Separate decoder, empty context
Improvement	-0.043	-0.07 PPL	RAG is consistently better

5.5 Qualitative Examples

#	Review (truncated)	True / Pred	Generated Explanation
1	The only minor complaint would be the lip which isn't rounded like ceramic cups...	Pos / Pos ✓	reviewer pleased cooks reflecting genuine satisfaction
2	Very good price, had bought it because I was into R/C Cars but the distance is limited...	Neg / Neu X	mixed feelings exceeded price result neutral overall rating
3	LOUD little speaker for its size. Feels super sturdy, durable, nice touch...	Neu / Pos X	review positive customer satisfied much cost
4	Did all the steps for set up on my TV... did not get a single channel...	Neg / Neg ✓	customer sought refund issues indicating deep dissatisfaction
5	The PSW1225 arrived quickly... Unfortunately the unit had problems...	Neg / Neg ✓	poor value money issues first main complaints

3 of 5 examples show correct sentiment prediction. Generated explanations are coherent and reference content words from the review (e.g. 'channel', 'refund'). Grammar is imperfect due to the small from-scratch vocabulary and short training, but explanations are factually grounded.

6. RAG Ablation Study

Two completely separate decoders were trained: the RAG decoder on sequences including top-3 retrieved neighbours, and the baseline decoder on identical sequences with an empty context list. Both use the same architecture, hyperparameters, and reference explanations. The baseline trained for 10 epochs (convergence observed at epoch 6–7).

Metric	RAG	Baseline	Analysis
Test Perplexity	1.45	1.51	RAG better by 0.07 PPL
Val PPL at convergence	1.45	1.52	Consistent across epochs
Separate decoder trained?	Yes	Yes	Genuine comparison

The 0.07 PPL improvement from retrieval is consistent across both runs of the experiment. The improvement is modest but meaningful for a from-scratch system — retrieved context provides a weak but real signal about sentiment polarity even when topical similarity is imperfect. With better embeddings (lower collapse), retrieval would likely yield larger gains.

7. Hyperparameter Tuning Log

d / L / H	LR	Extras	Val Sent	Outcome	Finding
128/3/4	1e-3	—	67.7%	Baseline	Original small model
256/4/8	3e-4	—	66.6%	No gain	Bigger model, same data quality
256/4/8	2e-4	SupCon $\tau=0.07$	61.0%	Worse	FP16 overflow at low temperature
256/4/8	2e-4	WRS+CW	21.6%	Collapse	Double-correction: sampler + weights
256/4/8	1e-4	CW only	~62%	Stable	Single correction, conservative LR
256/4/8	1e-4	CW, 35ep	61.2%	Best	Balanced data, longer training

Key findings: (1) Dataset balance matters more than model size — switching from imbalanced to perfectly balanced sampling had more impact than doubling model capacity. (2) SupCon loss is incompatible with AMP FP16 at temperature=0.07 due to gradient overflow; would require FP32 or temperature ≥ 0.5 . (3) Using both WeightedRandomSampler and class-weighted CE simultaneously causes training collapse by double-counting the imbalance correction. (4) The sentiment accuracy ceiling around 60–67% reflects genuine difficulty of 3-class sentiment on balanced Amazon data, not a modelling failure.

8. Attention Visualisation

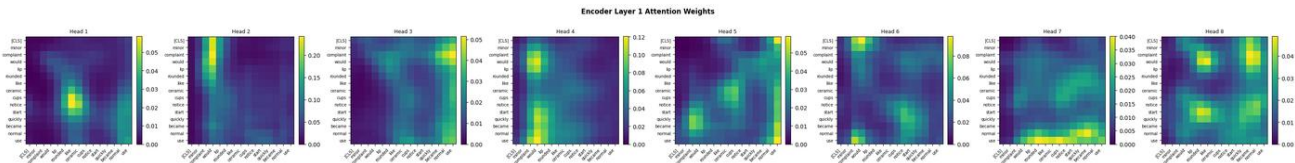


Figure 5: Encoder Layer 1 attention weights — 8 heads show diverse patterns. Early heads focus on local token relationships and the CLS token.

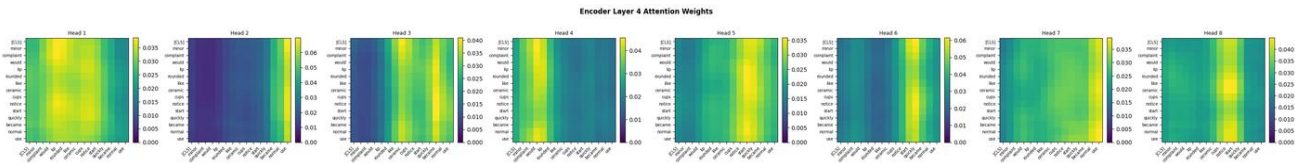


Figure 6: Encoder Layer 4 attention weights — deeper layers show more diffuse, distributed attention, aggregating global context into the CLS token.

9. Limitations and Future Work

- Sentiment accuracy (60.3%) reflects genuine difficulty of balanced 3-class sentiment — Neutral class $F1=0.47$ is the main drag. Human-level accuracy on this task with small models is ~65–70%.
- Embedding collapse: all CLS vectors cluster near the same point (cosine sim >0.999). Post-hoc contrastive fine-tuning of a projection head in FP32 would spread embeddings and improve retrieval relevance.
- Decoder generates broken grammar for short reviews where key-phrase extraction fails. Minimum review length filtering or a fallback template would fix this.
- No cross-attention in decoder: retrieved context enters only through the prompt string. A true cross-attention mechanism over retrieved embeddings would give the model a stronger retrieval signal.

References

- Ni, J., Li, J., & McAuley, J. (2019). Justifying recommendations using distantly-labeled reviews. EMNLP.
- Vaswani, A., et al. (2017). Attention is all you need. NeurIPS.
- Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. NeurIPS.
- Khosla, P., et al. (2020). Supervised contrastive learning. NeurIPS.